

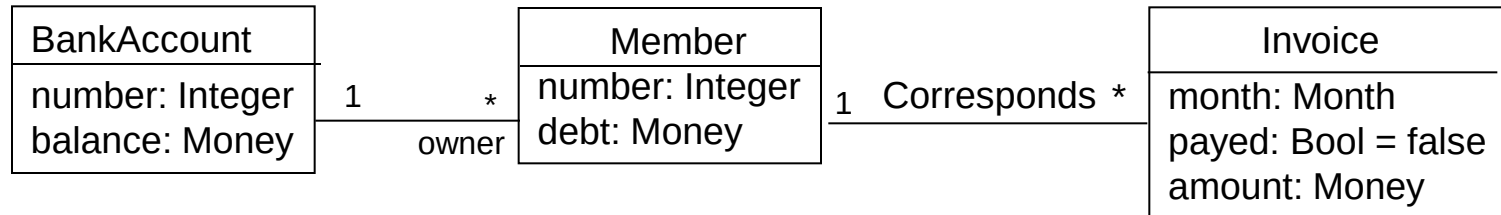
Data Layer Design

Example

Example

- Specification Example
- Design without considering persistence aspects
- Design - automatic generation of persistence
- Design - direct design of persistency
 - Controller as pure intermediary (Stored Procedures)
 - Row Data Gateway
 - Active Record

Specification Example



context DomainLayer::payInvoice (ns: Integer, m: Month)

-- operation for paying the Invoice of month *m* from Member *ns*

exc invoice-not-exists: it does not exist an Invoice from Member *ns* in month *n*

invoice-payd: the invoice is already payd

low-balance: the member has not balance enough in its bank account

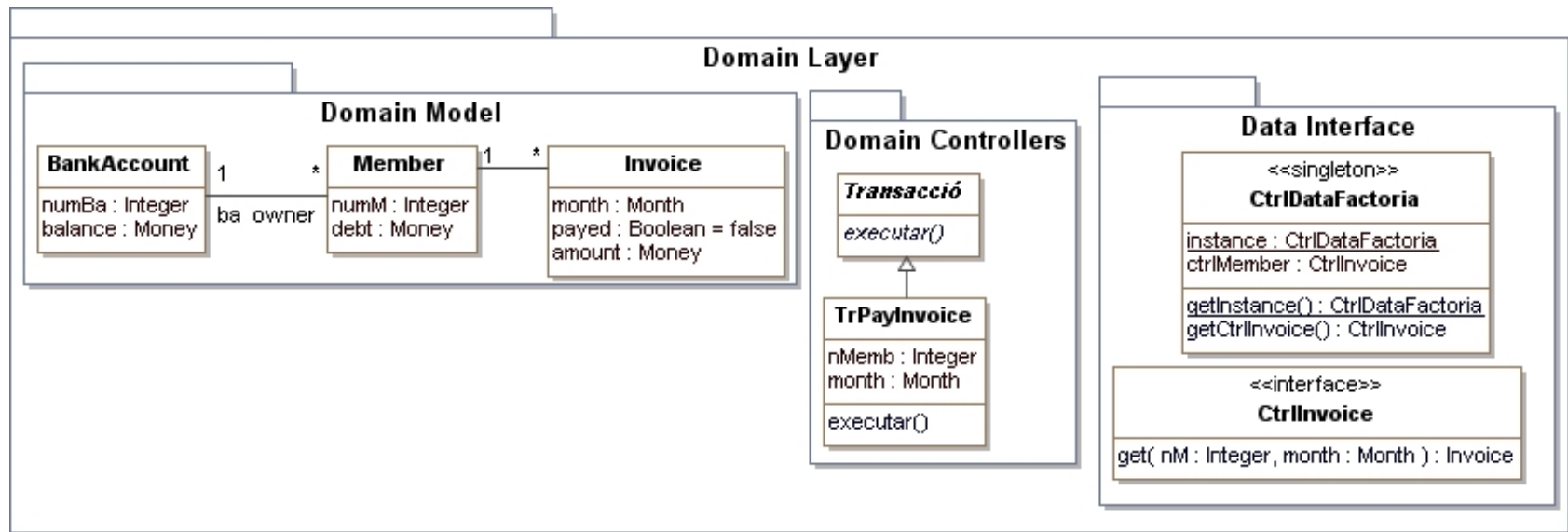
post:

2.1 The attribute *payd* from *Invoice* is set to true

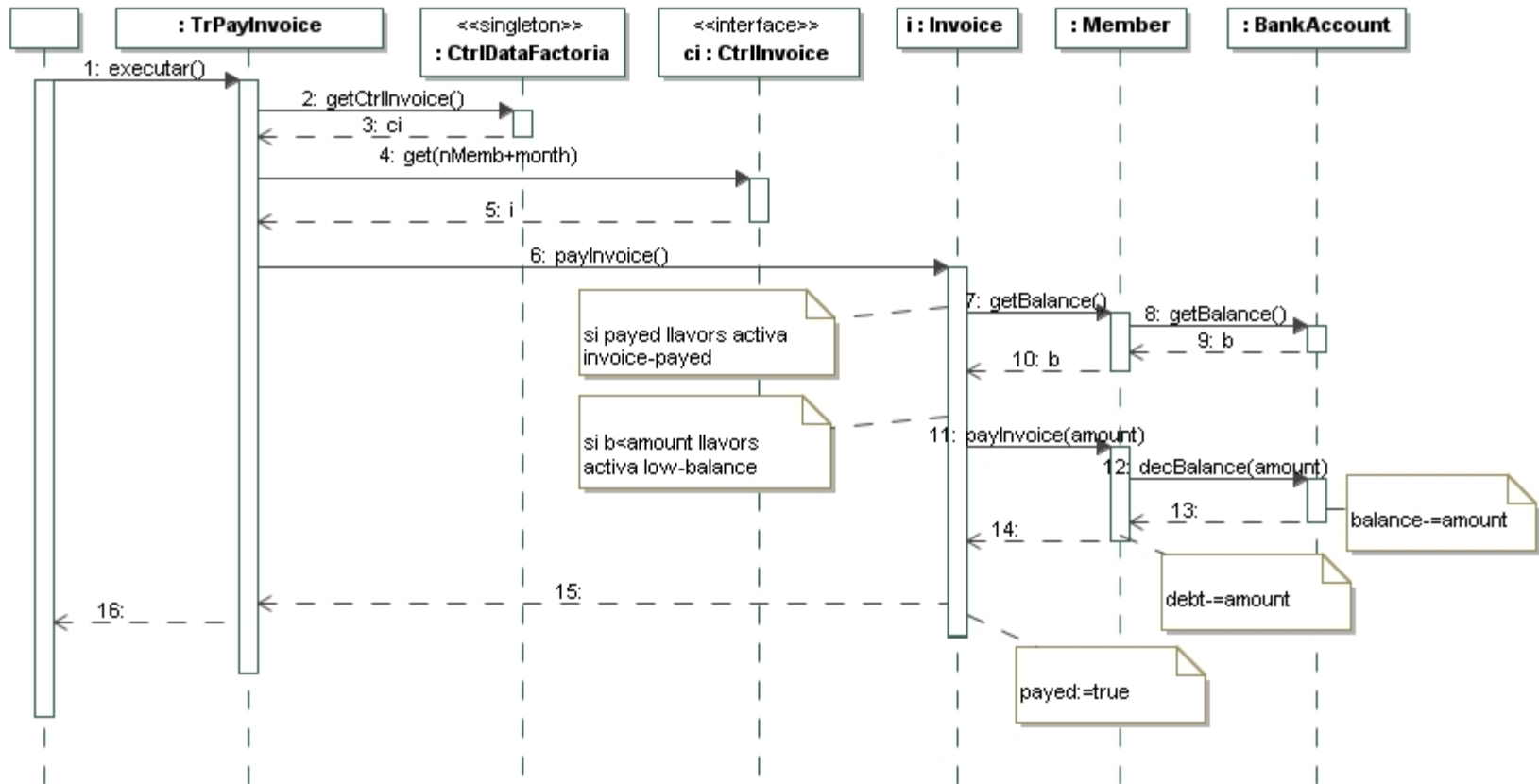
2.2 Balance of Member's BankAccount is decremented with Invoice's amount

2.3 Member's debt is decremented with Invoice's amount

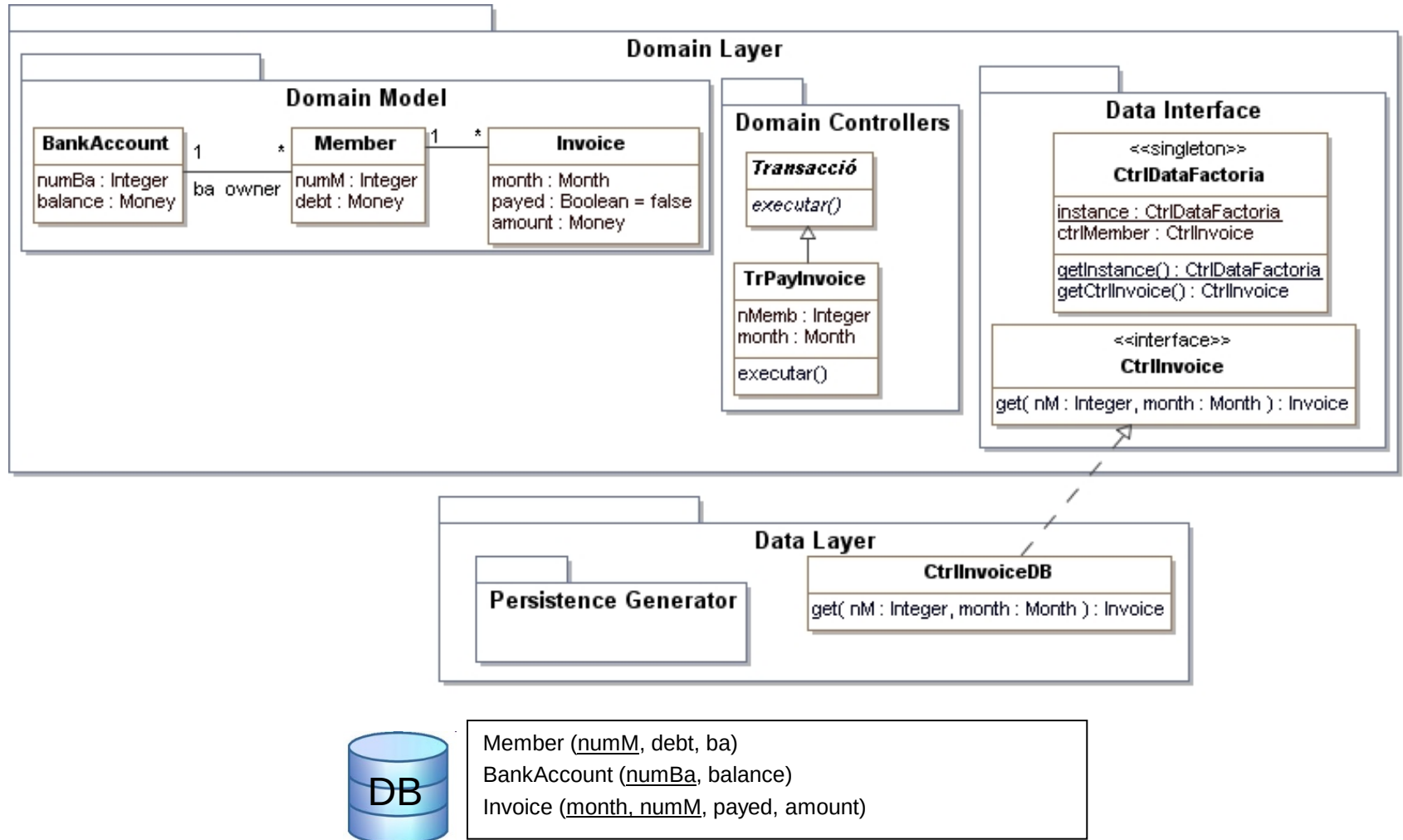
Design without considering persistence aspects



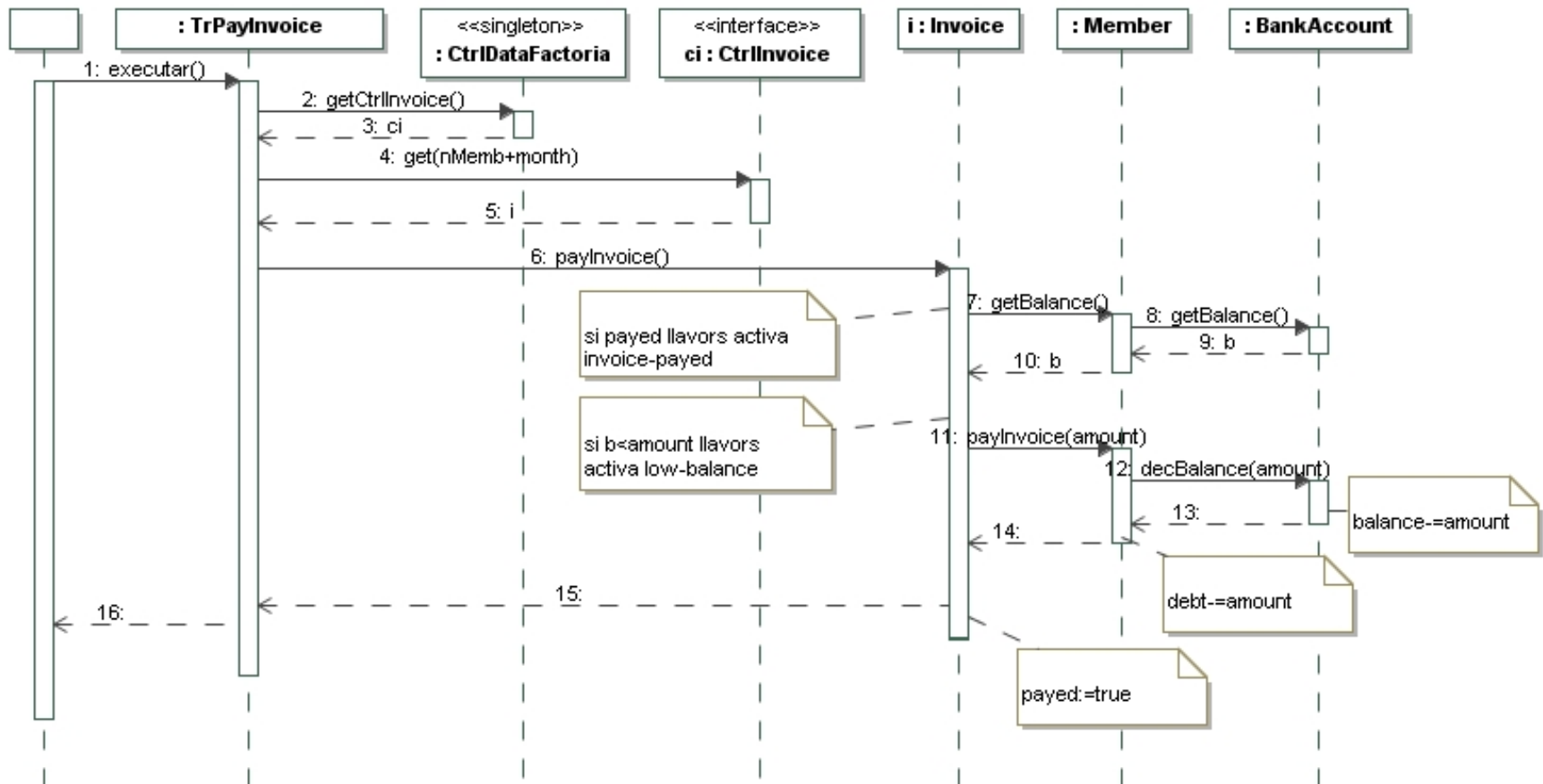
Design without considering persistence aspects



Design - automatic generation of persistence

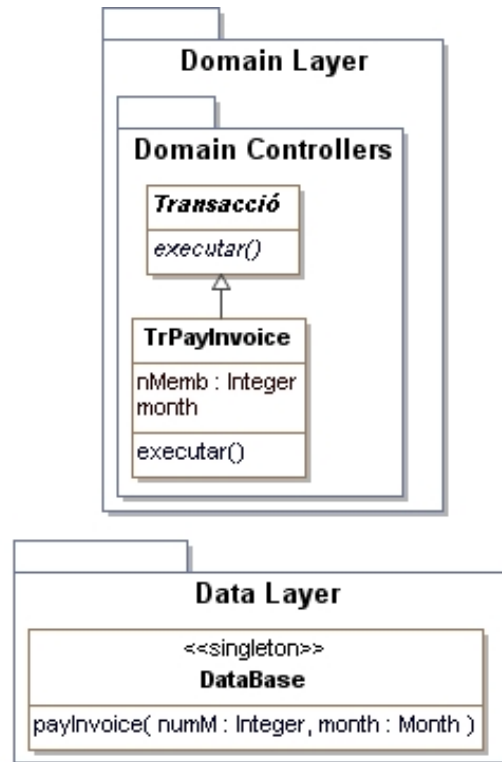


Design - automatic generation of persistence



Design – direct design of persistence

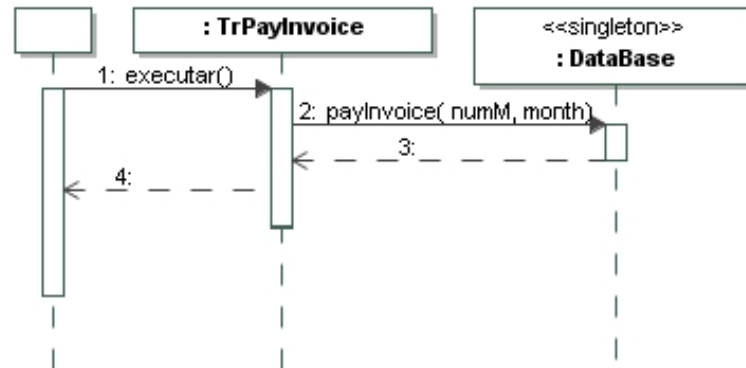
Domain Controller as pure intermediary



Member (numM, debt, ba)
BankAccount (numBa, balance)
Invoice (month, numM, payed, amount)

Design – direct design of persistence

Domain Controller as pure intermediary (stored procedure)

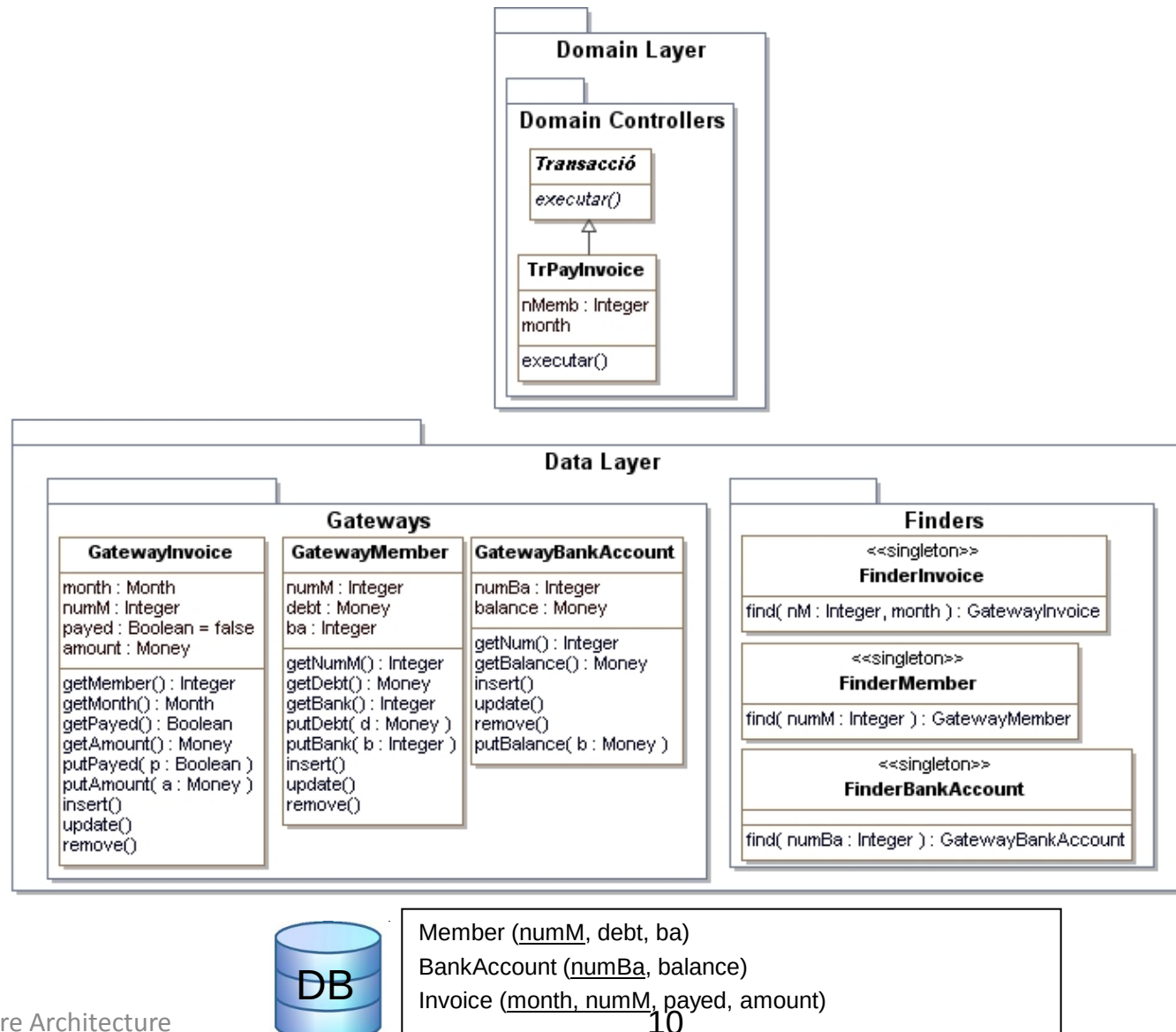


```
CREATE PROCEDURE PayInvoice (numMemb, monthTramitation)
RETURNING INTEGER, CHAR(50);
DEFINE error-code INTEGER; ....
ON EXCEPTION SET error-code, error-miss; RETURN error-code, error-miss END EXCEPTION;

IF ((SELECT COUNT(*) FROM invoice WHERE month=monthTramitation AND member=numMemb)=1) THEN
    LET amount, l-paid = (SELECT amount, paid FROM invoice WHERE month=monthTramitation AND member=numMemb));
    IF ('Y' = l-paid ) THEN RAISE EXCEPTION 2, 'The invoice is already payed';
        ELIF LET balance, numba = (SELECT c.balance,c.num-ba FROM member s, bankaccount c WHERE s.ba=c.num-ba
                                and s.numM=numMemb);
            IF balance < amount THEN RAISE EXCEPTION 3, 'The member has not balance enough';
            ELSE UPDATE invoice SET paid = 'Y' WHERE month=monthTramitation AND member=numMemb;
                UPDATE member s SET s.debt = s.debt-amount WHERE s.numM=numMemb;
                UPDATE bankaccount c SET c.balance = c.balance-balance WHERE num-ba =c.num-ba ENDIF
        ELSE RAISE EXCEPTION 1, 'The member has not invoices this month'; END IF;
    RETURN 0,'PayedInvoice';
END PROCEDURE;
```

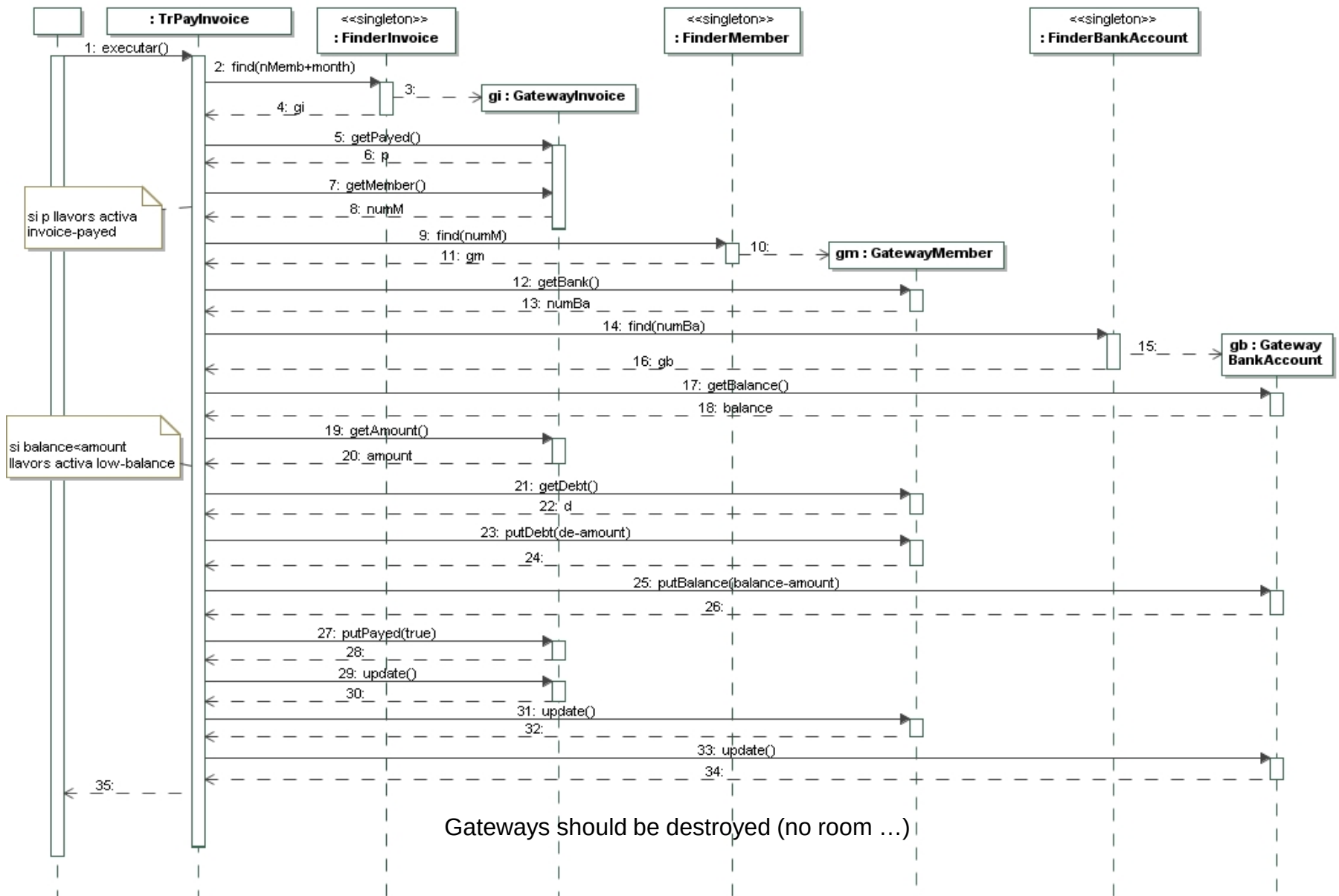
Design – direct design of persistence

Domain Controller with plain logic, Row Data Gateway pattern



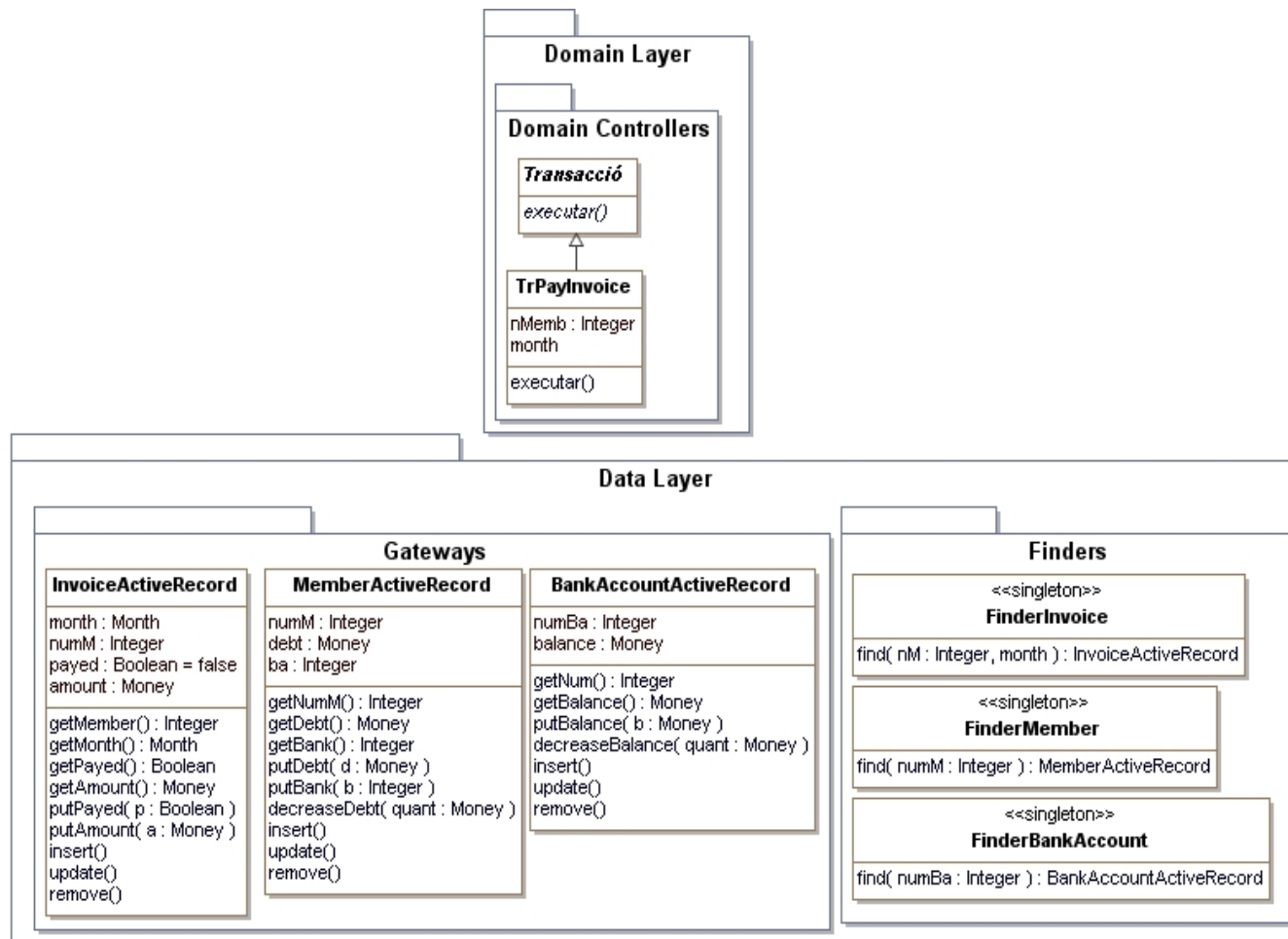
Design – direct design of persistence

Domain Controller with plain logic, Row Data Gateway pattern



Design – direct design of persistence

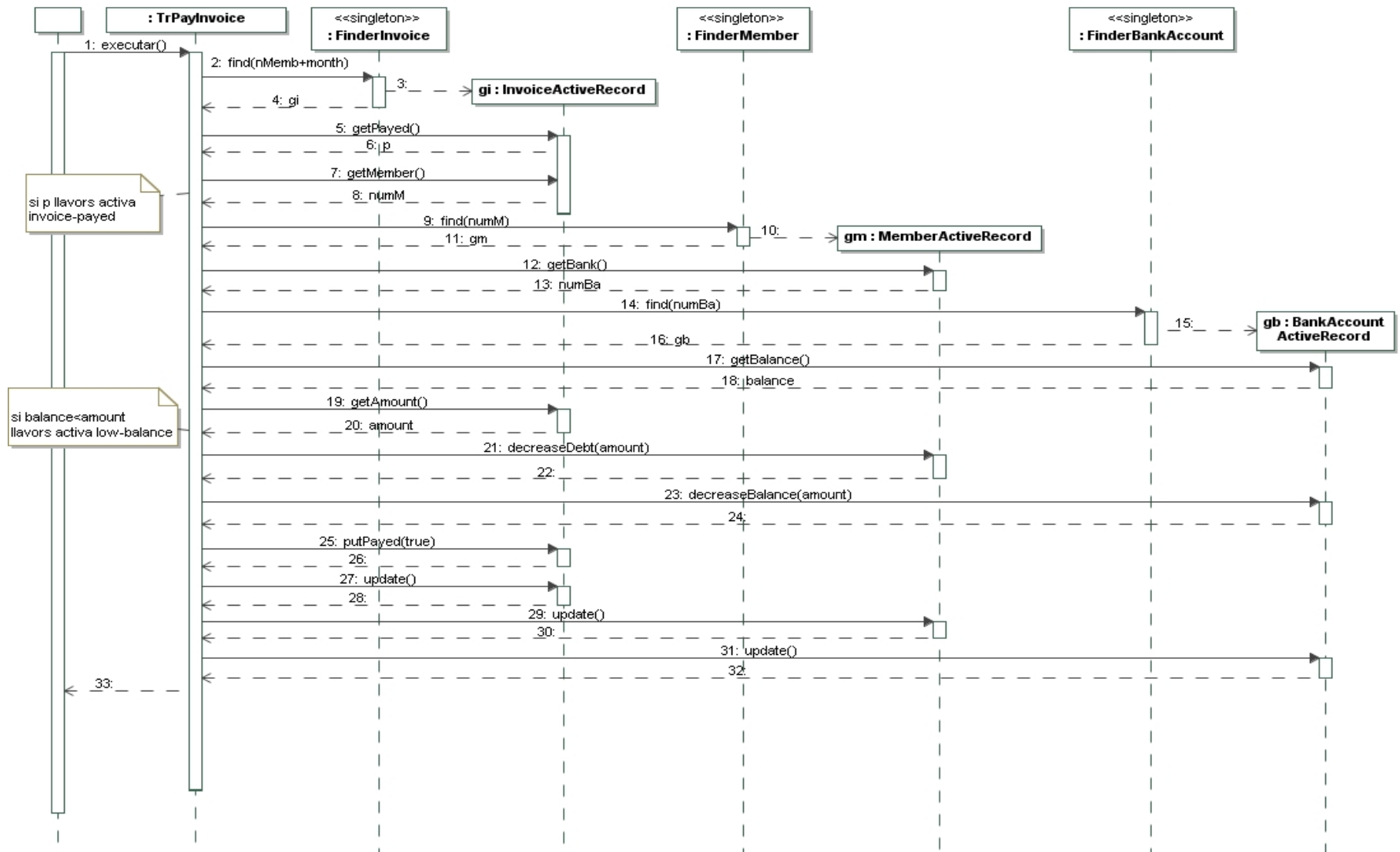
Domain Controller with plain logic, Active Record pattern



Member (numM, debt, ba)
BankAccount (numBa, balance)
Invoice (month, numM, payed, amount)

Design – direct design of persistence

Domain Controller with plain logic, Active Record pattern



References

- M. Fowler
Patterns of Enterprise Application Architecture
Addison-Wesley, 2003
- H. Garcia-Molina, J. Ullman, J. Widom
Database Systems Implementation
Prentice-Hall, 2000.
- J. Melton, A. Eisenberg
Understanding SQL and Java Together
Morgan-Kaufmann, 2000.
- Christian Bauer, Gavin King
Hibernate in Action
Manning Publications Company, 2004